

Discovery Scanner Findings — Daily Triage Workflow

A daily workflow for reviewing automated discovery-scan output — and the dashboard we want next.

Document	Discovery Scanner Findings — Daily Triage Workflow
Audience	Research team / Core testers covering triage
Owner	Discovery Agents / Research
Status	Working draft — process proposal

1. Purpose & The Ask

Discovery agents run automated security scanners against client targets on a fixed schedule. Scans kick off at **midnight on the day a test starts**, and we do not know which of them will actually succeed until they have finished running. With only a small number of testers actively triaging output, a lot of completed scan results currently sit unreviewed.

The ask is simple and low-cost: **any morning, a researcher can open the list of scans, look at the ones that completed successfully, and read through the scanner findings they produced.** You do not need to be assigned to the pentest, and you do not need to take formal action. The goal is a quick, informed pass over fresh output so that real issues surface faster and obvious noise is flagged for whoever owns the engagement.

Guardrail — do not formally reject or accept findings during this pass. Read the output, sanity-check it, and record a non-binding assessment for the tester who owns the engagement. Accept/reject decisions stay with the assigned tester.

What this document covers

- How scans run and why triage is reactive (Section 2).
- The step-by-step morning triage workflow (Section 3).
- The tools and sites in scope and how to think about coverage (Section 4).
- Reading a scan log to tell success from failure — worked example (Section 5).
- Anatomy of a single scanner finding — worked example (Section 6).
- A non-binding assessment rubric (Section 7).
- The dashboard we want eventually (Section 8).

2. How Discovery Scans Run

Each scan is a scheduled, automated job tied to a pentest. The scheduler launches the configured tools at midnight on the engagement's start day, and the jobs run unattended to completion (or failure). Because they run overnight and unattended, the first time anyone knows whether a scan produced usable output is the next morning — which is exactly the window this workflow targets.

A scan moves through a small set of states:

Status	Meaning	Triage action
Completed	The job finished and produced output / findings.	Primary target for review — read the findings.
Failed	The job exited with an error before finishing.	Usually skip; note repeat failures (may still hold partial passive output).
In progress	The job is still running.	Leave it; re-check later in the day.

*Treat the scan list as a worklist, not a guarantee. Roughly a third of scans fail on any given morning (timeouts, scope/context errors, target down). Sort to the **Completed** scans first — that is where the findings are.*

3. The Morning Triage Workflow

A full pass over a morning's successful scans should take well under an hour. Work the list top to bottom and keep notes as you go.

Step 1 — Open the list and filter to fresh, successful scans

1. Open the scans dashboard and set the date filter to today (or the morning you are reviewing).
2. Filter / sort by **Status = Completed**. These are the scans that produced output.
3. Optionally narrow by Tool or by site if you only want to cover one area.

Step 2 — Open each completed scan's output

1. For each completed scan, open its findings (and, when present, the raw tool log / report artifact).
2. Note the tool, the target site, and how many findings were produced.

Step 3 — Read and sanity-check each finding

1. Confirm the affected target is actually in scope for the engagement.
2. Check that the evidence backs the claim (e.g., the version string or response the tool cites is really present).
3. Decide whether it looks real and impactful, needs hands-on verification, or is likely informational / noise.

Step 4 — Record a non-binding assessment

1. Tag each finding with one of the assessment buckets in Section 7.
2. Write one or two lines of reasoning — enough for the assigned tester to act on quickly.
3. Do **not** click accept or reject. That decision stays with the engagement owner.

Step 5 — Hand off

1. Leave your notes where the assigned tester will see them, or flag anything that looks high-impact.
2. Move on to the next completed scan.

Output of the pass: a short, per-finding note ("likely valid / verify / noise" + one line) for the morning's completed scans. Nothing is formally triaged — you have just made the owner's job faster.

4. Tools & Sites In Scope

Discovery currently centers on three main scanning tools running against three main sites. Knowing what each tool is good (and bad) at tells you how much weight to give a finding before a human has looked at it.

The three tools

Tool	Category	What it produces	How much to trust unverified
ZAP	Vulnerability (DAST)	Spiders the app, runs passive + active web checks, retire.js library checks; exports a SARIF report.	Passive findings are reliable signal; active findings vary — verify impact.
Nuclei	Vulnerability (templates)	Matches targets against community/templated CVE & misconfig signatures.	High-confidence when a template matches exactly; watch for version-only matches.
ffuf	Reconnaissance (fuzzing)	Content / directory discovery — enumerates paths, params, vhosts.	Recon, not vulns — use it to understand surface, confirm interesting hits.

A recon-consolidation data-processor step also runs to merge discovered URLs into the input list the scanners consume (for example, the `consolidated_urls.txt` that ZAP imports). It is plumbing, not a findings source.

The three sites

Site	Role	Notes for triage
<code>app.petquant.com</code>	Primary web application	Main user-facing app; expect the bulk of web findings.
<code>api.petquant.com</code>	API surface	Endpoint/auth findings; cross-check against the app's behavior.
<code>staging-portal.givingafoundation.app</code>	Staging portal	Non-production; weigh impact accordingly (isolation from prod data).

Coverage at a glance

Site \ Tool	ZAP	Nuclei	ffuf
<code>app.petquant.com</code>	Vuln scan	Vuln scan	Recon
<code>api.petquant.com</code>	Vuln scan	Vuln scan	Recon
<code>staging-portal.givingafoundation.app</code>	Vuln scan	Vuln scan	Recon

Use the matrix to spot gaps: if a site has no Completed scan for a given tool this morning, that is a coverage hole to note (often the result of a failed overnight run).

5. Reading a Scan Log — Worked Example (ZAP)

Before trusting a scan's findings, confirm the scan actually did what it was supposed to. Tool logs make this quick. Below is an annotated walk-through of a real ZAP run against `app.petquant.com`.

Startup — confirm the tool and rule set loaded

```
INFO CommandLineBootstrap - ZAP 2.17.0 started 24/06/2026, 12:16:08 ...
INFO ScanRuleManager - Loaded passive scan rule:
    Vulnerable JS Library (Powered by Retire.js)
```

This tells you the engine version and that the retire.js passive rule is active — the rule that produces the “Vulnerable JS Library” finding examined in Section 6.

Jobs — see what ran and what it targeted

```
INFO CommandLine - Job import set fileName = /tool/consolidated_urls.txt
INFO CommandLine - Job Report set template = sarif-json
INFO CommandLine - Job Report set reportFile =
    zap-report-app.petquant.com-2026-06-24.sarif.json
INFO CommandLine - Job General spider found 8 URLs
```

The spider seeded from the consolidated URL list and discovered 8 URLs, and the run is configured to emit a SARIF report. So far, so good.

The failure — why this scan shows as “Failed”

```
ERROR ExtensionAutomation - The starting URI does not belong
    to the context: https://app.petquant.com
INFO CommandLine - Automation plan failures:
INFO Control - Automation Framework setting exit status to 1
    due to plan errors
```

The Ajax Spider job failed with a context/scope error, which set the overall exit status to 1 — so the orchestrator marks the scan **Failed** even though the traditional spider and passive scan ran first. Two takeaways:

- **Failed ≠ clean:** A Failed status is often a scope/context or timeout problem, not a target being secure.
- **Partial data:** Earlier jobs (spider, passive/retire.js) may still have produced partial output worth a glance, but for the morning pass, prioritize cleanly Completed scans first.

Quick log checklist: (1) did the expected tool/version start? (2) did the spider/import find URLs? (3) what is the final exit status / were there plan failures? (4) was the report artifact written? Answering these takes seconds and tells you how much to trust the output.

6. Anatomy of a Scanner Finding — Worked Example

This is a real finding produced by ZAP's retire.js passive rule and written up on the engagement. Use it as a model for what “good” scanner output looks like and what to check when you read one.

Vulnerable JS Library — Outdated Next.js

Field	Value
Title	Vulnerable JS Library (Outdated Software Version)
Vulnerability type	Components with Known Vulnerabilities > Outdated Software Version
CWE	CWE-1395: Dependency on Vulnerable Third-Party Component
OWASP severity	Low (Likelihood 2/5, Business Impact 2/5)
Affected target	https://staging-portal.givingafoundation.app
Affected resource	/_next/static/chunks/main-6efffc813d6db951.js
Detected version	Next.js 15.5.12
Source	ZAP passive rule — Vulnerable JS Library (Powered by Retire.js)

Description

The application bundles an outdated version of the Next.js framework (15.5.12), which is subject to multiple published CVEs. The version string is observable directly in the client-side JavaScript bundle. Because the issue is a dependency on a vulnerable third-party component (CWE-1395), exact exploitability depends on how the framework is used in the app.

Associated CVEs

CVE-2026-44580, CVE-2026-44581, CVE-2026-44582, CVE-2026-45109, CVE-2026-44576,
CVE-2026-29057, CVE-2026-44577, CVE-2026-44578, CVE-2026-44579, CVE-2026-44572,
CVE-2026-27980, CVE-2026-44573, CVE-2026-44574, CVE-2026-44575

Proof of concept — retrieve the bundle and observe the embedded version string:

```
GET /_next/static/chunks/main-6efffc813d6db951.js HTTP/1.1
Host: staging-portal.givingafoundation.app

HTTP/1.1 200 OK
Content-Type: application/javascript; charset=UTF-8
...
let H = "15.5.12"; // Next.js version embedded in the bundle
```

Suggested fix (as written on the finding)

- Upgrade Next.js to a patched release — latest 15.x (e.g. 15.5.18+) or 16.x (e.g. 16.2.6).
- Run `npm install / npm audit` after the bump and regression-test auth (`@auth0/nextjs-auth0`) and SSR paths.
- Longer term: SBOM generation, automated dependency monitoring (Dependabot/Renovate/Snyk), and lockfile pinning.

7. Assessing a Finding (Non-Binding)

For each finding, answer a few quick questions, then drop it into one of three buckets. Again: these are notes for the engagement owner, **not** accept/reject decisions.

Questions to ask

- Is the affected target actually in scope for this engagement?
- Does the cited evidence really support the claim (version string, response body, header)?
- Is the impact real and reachable, or is this default/informational?
- Is this a known false-positive class for the tool, or environment-specific (e.g. staging)?
- Could it chain with anything else in the same scan to raise impact?

Assessment buckets

Bucket	Use when	Note to leave
Likely valid	Evidence is concrete and the issue is plausibly real.	Why it looks real + suspected severity.
Needs verification	Plausible but needs hands-on confirmation of impact.	What to test to confirm/deny.
Likely noise / informational	Default config, out-of-scope, or known FP pattern.	Why it can probably be deprioritized.

Applying it to the Next.js example

The version string is directly observable in the served bundle, so the detection itself is **Likely valid** — the dependency really is outdated. However, it is a dependency-only finding on a non-production staging portal and is rated Low, so the practical impact hinges on whether any of the cited CVEs are actually reachable in this app. A good non-binding note would be:

“Likely valid detection (Next.js 15.5.12 confirmed in `main-*.js` on staging portal). Low impact as written — dependency-only, non-prod. Worth a quick check of whether any cited CVE is reachable via the app’s actual usage before the owner prioritizes the upgrade.”

8. The Dashboard We Want Eventually

Triage is only as good as the visibility into which scans ran and which succeeded. The target is a single “Discovery Agents Health” view that a reviewer opens each morning to drive the workflow in Section 3.

Top-line health tiles

Scans Count	Scans Failed	Error Rate	Avg Runtime	Slowest
3	0	33.33%	01:09	69m

At-a-glance counts so a reviewer instantly knows the size and health of the morning's batch.

Filters — the view should be sliceable by the dimensions a reviewer actually triages along:

- **Org:** Org slug (which client).
- **Date:** default to today.
- **Pentest:** pentest tag (which engagement, e.g. #PT39025).
- **Tool:** ZAP / Nuclei / ffuf.

Scans detail table — a row per scan with the fields needed to decide what to open first:

org_slug	pentest_tag	level	scan_status	runtime_min	created_at
cortechsai	#PT38928	low	failed	null	Jun 24, 12:04 PM
finzly	#PT39135	high	in_progress	null	Jun 24, 12:04 PM
givinga	#PT39025	high	completed	69	Jun 24, 11:58 AM

Plus scan_started_at and scan_completed_at columns. The completed, high-level givinga scan is exactly the kind of row a reviewer should click into first.

Findings list (drill-down) — clicking a scan should reach a findings/scan list with at least:

Column	Purpose
Tool	Which scanner produced the row (zap, nuclei, ffuf).
Type	vulnerability / reconnaissance / data_processor.
Scan start / Scan end	When it ran — freshness and runtime.
Status	Completed / Failed — the filter that drives triage.
Actions / Download	Open findings or pull the raw report artifact (e.g. SARIF, logs).

Requirements summary

1. One morning view that lists every scheduled scan and its status, filterable by org, date, pentest, and tool.
2. A clear Completed-vs-Failed signal so reviewers can jump straight to reviewable output.
3. One-click access from a scan to its findings and its raw artifact (report + tool log).
4. Health metrics (count, failed, error rate, runtime) to track scanner reliability over time.
5. A lightweight place to leave non-binding triage notes per finding for the engagement owner.

Appendix A — Glossary

Term	Meaning
ZAP	OWASP Zed Attack Proxy — dynamic web app scanner (DAST).

Term	Meaning
Nuclei	Template-based vulnerability scanner matching signatures to targets.
ffuf	"Fuzz Faster U Fool" — content/path discovery fuzzer (reconnaissance).
retire.js	Library that flags JavaScript dependencies with known vulnerabilities; runs as a ZAP passive rule.
Passive vs active scan	Passive observes existing traffic/responses; active sends crafted probes. Passive findings carry less risk of FPs.
SARIF	Static Analysis Results Interchange Format — the JSON report format ZAP emits.
CWE-1395	Dependency on a vulnerable third-party component.
Pentest tag	Engagement identifier, e.g. #PT39025 .

Appendix B — References

- Discovery Agents Health dashboard (Looker Studio) — morning scan status view.
- Engagement findings view — per-pentest scanner output and triage.
- Vercel / Next.js security advisories — detail for the CVEs cited in Section 6.
- Tool docs: OWASP ZAP, Nuclei (ProjectDiscovery), ffuf — capabilities and report formats.

Working draft — process proposal for scaling daily scanner-findings triage.